

Bob 1

Chuqur o'rganishga kirish va sun'iy neyron

Tayanch manbalar: [1, 5].

M1 va S1: o'rganiladigan savol

Bir nechta son yordamida tasvir, matn yoki signal uchun qaror chiqarish mumkinmi? Agar qaror chegarasi to'g'ri chiziq bo'lmasa, bitta neyron nimaga ojiz qoladi? Bob oxirida talaba shu chegarani matematik ko'rsatadi, faollashtirish hosilasini chiqaradi va XOR masalasini avtomatik differensiallashsiz yechadi.

1.1 Belgini qo'lda yozishdan belgini o'rganishga

Qo'lyozma raqamni tanishda uning qalinligi, qiyaligi va joylashuvi o'zgaradi. Har bir holat uchun alohida shart yozish qoidalar sonini oshiradi. O'rganiladigan model esa $f_{\theta}(x)$ funksiyaning parametrlarini misollar asosida moslaydi. Chuqurlik bir nechta o'rganiladigan akslantirishning kompozitsiyasidir:

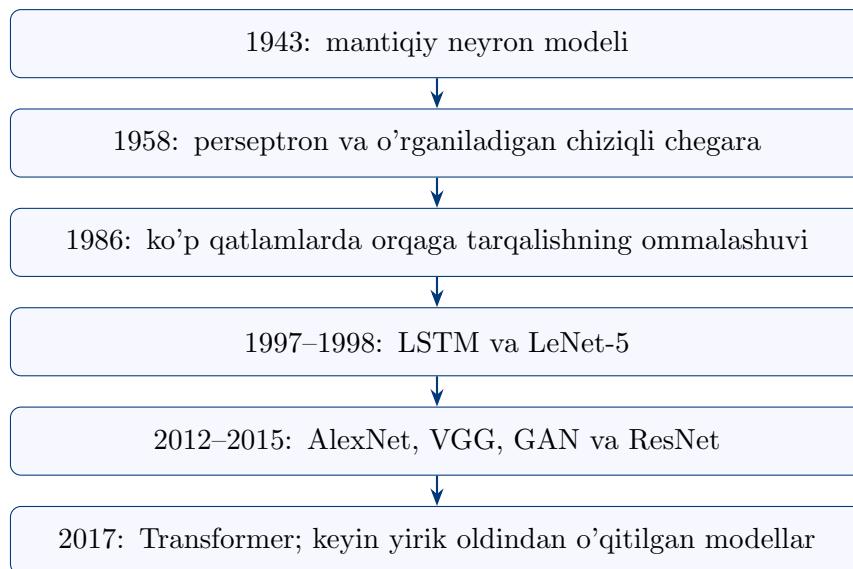
$$f_{\theta}(x) = f_L \circ f_{L-1} \circ \dots \circ f_1(x).$$

Tasvirda dastlab lokal farqlar, keyin konturlar, so'ng murakkab shakllar uchun foydali belgilar paydo bo'lishi mumkin. Bu talqin har bir neyron albatta inson tushunadigan tushunchani ifodalaydi degani emas. O'qitish algoritmi insonning izohini emas, berilgan yo'qotishni kamaytiradi.

Klassik usulda $\phi(x)$ belgi muhandis tomonidan tanlanib, yakuniy $g_w(\phi(x))$ o'rganiladi. Neyron tarmoqda ϕ_{θ} ham moslashtiriladi. Yetarlicha kichik jadval ma'lumotida chiziqli model yoki daraxtlar ansambli kuchli boshlang'ich mezon bo'lishi mumkin. Arxitektura tanlashda ma'lumot hajmi, tuzilishi, hisoblash xarajati va tushuntirish ehtiyoji birga qaraladi. Chuqurlikning o'zi sifat kafolati emas.

1.2 Tarix: har bir bosqich qaysi to'siqni yengdi?

Sun'iy neyron tarixi mantiqiy hisoblash, parametr o'rganish va katta hajmdagi hisoblashning birlashishidir. McCulloch–Pitts modeli mantiqiy elementni neyron orqali ifodaladi; perseptron og'irliklarni moslash qoidasi bilan bog'landi. Ko'p qatlamlarni amaliy o'qitish uchun zanjir qoidasi asosidagi orqaga tarqalish muhim bo'ldi. CNN lokal bog'lanish va umumiy og'irliklardan, LSTM esa boshqariladigan xotiradan foydalandi. GPU va yirik belgilangan to'plamlar avvalgi g'oyalarni kattaroq miqyosda sinashga imkon berdi.



Bu vaqt chizig'i yagona kashfiyotlar ketma-ketligi emas; tanlangan tayanch bosqichlarni ko'rsatadi. Masalan, avtomatik differentsiallash va zanjir qoidasining tarixi 1986-yildan oldin ham mavjud. Tarixiy sanani usulning barcha ko'rinishlari bir vaqtda yaratilgan degan da'voga aylantirmaslik kerak.

1.3 Neyronning skalar, vektor va paket modeli

1.3.1 Og'irlik va siljishning ma'nosi

Ikkita belgi asosida qaror chiqaraylik: x_1 signal kuchi va x_2 uning davomiyligi. Ularning hissalari w_1x_1, w_2x_2 bilan qo'shiladi; siljish chegarani koordinata boshidan suradi:

$$z = \sum_{j=1}^d w_j x_j + b = w^\top x + b, \quad a = f(z).$$

Biologik neyronning kirishlarni yig'ishi bilan bu model orasida ilhomiy o'xshashlik bor. U ion kanallari yoki impuls vaqtlarini to'liq modellashtirmaydi. Bu kursda neyron differentsiallanuvchi hisoblash elementi sifatida qaraladi.

Misol 1.1 (Bitta neyronni hisoblash). $x = (2, -1)^\top$, $w = (0.5, 1)^\top$, $b = 0.2$ bo'lsa, $z = 0.2$. Sigmoid chiqish $1/(1 + e^{-0.2}) \approx 0.549834$, ReLU chiqish 0.2. Sigmoidni ehtimol sifatida talqin qilish uchun binar ehtimollik modeli va mos yo'qotish kerak. ReLU chiqishini bevosita ehtimol deyish mumkin emas.

Paketda $X \in \mathbb{R}^{B \times d}$, $W \in \mathbb{R}^{d \times h}$, $b \in \mathbb{R}^h$ bo'lsa,

$$Z = XW + \mathbf{1}_B b^\top \in \mathbb{R}^{B \times h}, \quad A = f(Z).$$

Kutubxonalarda siljish avtomatik kengaytiriladi. Bu yozuvda ustunlar chiqish neyronlariga mos; `nn.Linear` og'irlikni (h, d) shaklida saqlab, hisoblashda transponirlaydi. Nazariy W bilan dasturdagi `weight` shaklini adashtirish gradient xatolarining keng tarqalgan sababidir.

1.3.2 Nega nochiziqlilik shart?

Ikki affin qatlamni birlashtirsak,

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

Demak, faollashtirishsiz chuqurlik qaror funksiyasining affin tabiatini o'zgartirmaydi. Nochiziqlilik yangi funksiyalar oilasini hosil qiladi. Parametrlar ko'payishi va ifodalash qobiliyati bilan birga o'qitish va umumlashtirish muammosi ham kuchayadi.

1.4 Perseptron, ajraluvchanlik va XOR

Perseptron $\hat{y} = \mathbf{1}[w^\top x + b \geq 0]$ qarorini chiqaradi. Xato bo'lsa,

$$w \leftarrow w + \eta(y - \hat{y})x, \quad b \leftarrow b + \eta(y - \hat{y}).$$

Bu silliq yo'qotishning oddiy orqaga tarqalishi emas; xato tasniflangan nuqtani qaror chegarasining kerakli tomoniga suruvchi qoidadir. Chiziqli ajraluvchan chekli to'plam uchun, chegaraning musbat zaxirasi mavjud bo'lganda, standart perseptron yaqinlashish kafolatiga ega. Ajralmaydigan ma'lumot uchun bunday kafolat yo'q.

Tasdiq 1.1 (XOR uchun chiziqli chegara yetarli emas). $(0, 0)$, $(1, 1)$ *manfiy*, $(1, 0)$, $(0, 1)$ *musbat bo'lsin*. Barcha nuqtalarni qat'iy zaxira bilan ajratadigan affin neyron mavjud emas.

Isbot. Musbat nuqtalar uchun $w_1 + b > 0$ va $w_2 + b > 0$, demak $w_1 + w_2 + 2b > 0$. Manfiy nuqtalar uchun $b < 0$ va $w_1 + w_2 + b < 0$, shuning uchun ularni qo'shib $w_1 + w_2 + 2b < 0$ olamiz. Qarama-qarshilik hosil bo'ldi. $\square$

Ikki yashirin mantiqiy belgi orqali XORni ifodalash mumkin: “kamida bitta birlik” va “ikkalasi ham birlik emas”. Amaliy silliq tarmoq yashirin belgilarni o'zi moslashtiradi. Ifodalash mavjudligi gradient algoritmi har qanday boshlang'ich nuqtadan yechimga yetishini bildirmaydi.

1.5 Faollashtirish funksiyalarini chiqarish va tanlash

1.5.1 Sigmoid va uning to'yinishi

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z)).$$

$0 < \sigma < 1$ va $\sigma' \leq 1/4$. $|z|$ katta bo'lganda hosila nolga yaqinlashadi. Ko'p yashirin qatlamlarda bunday koeffitsiyentlar ko'payishi gradientni kichraytirishi mumkin. Binar chiqishda esa Bernulli modeli bilan birga ishlatiladigan sigmoid juda tabiiy tanlovdir. Dasturda sigmoid va logarifmni alohida hisoblash o'rniga logit asosidagi barqaror yo'qotish ishlatiladi.

1.5.2 Giperbolik tangens

$$\tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1, \quad (\tanh z)' = 1 - \tanh^2 z.$$

Chiqish nol atrofida simmetrik. Bu yashirin holat uchun foydali, lekin katta argumentdagi to'yinish yo'qolmaydi. $z = 0$ da hosila birga teng; $z = 3$ atrofida esa juda kichik. Kirishlarni me'yorlash faollashtirishni mazmunli diapazonda ushlab turishga yordam beradi.

1.5.3 ReLU va LeakyReLU

$$r(z) = \max(0, z), \quad r'(z) = \begin{cases} 0 & z < 0, \\ 1 & z > 0. \end{cases}$$

Nolda klassik hosila mavjud emas; avtomatik differensiallash ma'lum konvensiyani, odatda nolni oladi. Musbat tomonda to'yinmaslik gradient uzatilishini yengillashtiradi. Barcha kuzatuvlar uchun neyron kirishi manfiy bo'lib qolsa, gradient nol bo'ladi. Bu holat "o'chib qolgan ReLU" deb ataladi; juda katta qadam yoki siljish bunga sabab bo'lishi mumkin.

$$r_\alpha(z) = \max(z, \alpha z), \quad 0 < \alpha < 1.$$

LeakyReLU manfiy tomonda α hosilani saqlaydi. Bu barcha o'qitish muammolarini yo'qotmaydi; faqat nol gradientning bitta manbasini yumshatadi.

1.5.4 GELU

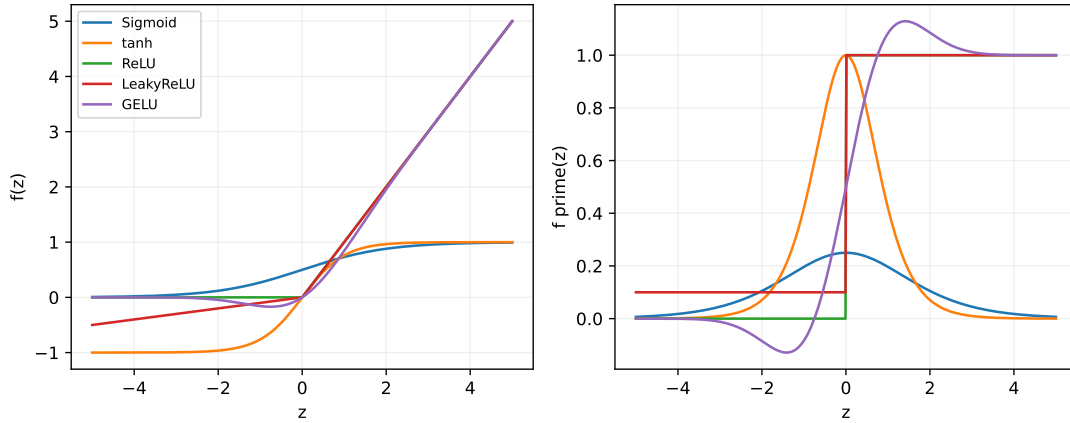
Standart normal taqsimot funksiyasi Φ orqali $g(z) = z\Phi(z)$ olinadi. Ko'paytma qoidasidan

$$g'(z) = \Phi(z) + z\varphi(z), \quad \varphi(z) = \frac{1}{\sqrt{2\pi}}e^{-z^2/2}.$$

GELU manfiy kirishni uzluksiz yumshatadi va silliq hosilaga ega. U barcha nuqtalarda monoton emas. Hisoblash ReLUDan qimmatroq bo'lishi mumkin; amaliy afzallik arxitektura va tajribaga bog'liq.

Funksiya	Chiqish va hosila	Foydali holat	Cheklov
----------	-------------------	---------------	---------

Sigmoid	$(0, 1), s(1 - s)$	Binari ehtimol, darvoza	Ikki tomonda to'yinish
tanh	$(-1, 1), 1 - a^2$	Cheklangan yashirin holat	Katta argumentda to'yinish
ReLU	$[0, \infty), 0/1$	CNN, oddiy MLP	Manfiy tomonda nol gradient
LeakyReLU	$\mathbb{R}, \alpha/1$	GAN diskriminatori	α tanlash kerak
GELU	Silliqlik, cheklanmagan yuqori tomon	Transformer FFN	Hisoblash xarajati, monoton emas



Rasm 1.1: Formulalardan hisoblangan faollashtirishlar va hosilalar. Noldagi ReLU hosilasi grafikdagi tanlangan konvensiyadir.

1.6 Og'irliklarni boshlashning matematik sababi

Mustaqil, nol o'rtachali kirishlar va og'irliklar uchun

$$\text{Var}(z) = \sum_{j=1}^d \text{Var}(w_j x_j) = d \text{Var}(w_j) \text{Var}(x_j).$$

Dispersiyani qatlamdan qatlamga saqlash uchun $\text{Var}(w) \approx 1/d$ kerak. ReLU nol atrofidagi simmetrik signalning ikkinchi momentini taxminan yarmiga tushiradi; shuning uchun He boshlashi $\text{Var}(w) \approx 2/d$ ni tanlaydi. Bu taxminlar mustaqillik va simmetriyaga tayanadi, o'qitish davomida aniq tenglik bo'lib qolmaydi. Barcha og'irliklarni bir xil nolga qo'yish yashirin neyronlarni simmetrik qiladi: ular bir xil gradient olib, farqli belgilarni o'rganmaydi.

1.7 S1: muhit, topshiriq, yondashuv va to'liq kod

1.7.1 Muhitni yaratish

Loyiha ildizida quyidagi buyruqlar bajariladi. Birinchi dasturga NumPy yetarli; qolganlari uchun qo'shimcha kutubxonalar kerak. `requirements-lock.txt` bajarilgan muhit versiyalarini saqlaydi.

```
1 py -3.12 -m venv .venv
2 .venv\Scripts\python.exe -m pip install -r requirements-lock.txt
3 .venv\Scripts\python.exe python/s01_numpy.py
```

Topshiriq. AND uchun perseptron va XOR uchun $2 \rightarrow 8 \rightarrow 1$ tarmoq yarating. **Yondashuv.** Dastlab chiziqli masalani ajrating, so'ng tanh yashirin qatlam va barqaror binar yo'qotishdan foydalaning. Hosilalar qo'lda hisoblanadi; barcha gradientlar eski parametrlar bilan olinib, keyin parametrlar yangilanadi.

```
1 """S1: perceptron and XOR network, no autograd and no torch modules."""
2 import json
3 from pathlib import Path
4 import numpy as np
5
6 def sigmoid(z):
7     return np.exp(-np.logaddexp(0, -z))
8
9 def run():
10     rng = np.random.default_rng(42)
11     x = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
12     and_y = np.array([0, 0, 0, 1])
13     w, b = np.zeros(2), 0.0
14     mistakes = []
15     for epoch in range(30):
16         errors = 0
17         for row, target in zip(x, and_y):
18             prediction = int(row @ w + b >= 0)
19             delta = target - prediction
20             w += .2 * delta * row
21             b += .2 * delta
22             errors += int(delta != 0)
23         mistakes.append(errors)
24         if errors == 0:
25             break
26     y = np.array([[0.], [1.], [1.], [0.]])
27     w1 = rng.normal(0, .7, (2, 8))
28     b1 = np.zeros((1, 8))
29     w2 = rng.normal(0, .7, (8, 1))
30     b2 = np.zeros((1, 1))
31     losses = []
32     for step in range(4000):
33         h = np.tanh(x @ w1 + b1)
34         z = h @ w2 + b2
35         p = sigmoid(z)
```

```

36     loss = np.mean(np.logaddexp(0, z) - y * z)
37     dz = (p - y) / len(y)
38     dw2, db2 = h.T @ dz, dz.sum(0, keepdims=True)
39     dh = (dz @ w2.T) * (1 - h * h)
40     dw1, db1 = x.T @ dh, dh.sum(0, keepdims=True)
41     w1 -= .2 * dw1
42     b1 -= .2 * db1
43     w2 -= .2 * dw2
44     b2 -= .2 * db2
45     losses.append(float(loss))
46     p = sigmoid(np.tanh(x @ w1 + b1) @ w2 + b2)
47     assert np.array_equal((p >= .5).astype(int), y)
48     result = {'seed': 42, 'and_weights': w.tolist(), 'and_bias': b,
49             'perceptron_epochs': len(mistakes), 'xor_probability': p.ravel().tolist()
50             ,
51             'first_loss': losses[0], 'last_loss': losses[-1], 'loss': losses}
52     out = Path(__file__).resolve().parents[1] / 'results'
53     out.mkdir(exist_ok=True)
54     (out / 's01.json').write_text(json.dumps(result, indent=2), encoding='utf-8')
55     print({k: v for k, v in result.items() if k != 'loss'})
56 if __name__ == '__main__':
57     run()

```

1.7.2 Natijani talqin qilish

Dastur AND uchun xatosiz epoxaga yetganda to'xtaydi. XORdagi to'rtta ehtimol 0, 1, 1, 0 yorliqlarga mos chegaraning to'g'ri tomonida bo'lishi tekshiriladi. Bu to'rtta nuqtani o'rganish dalilidir; yangi taqsimotga umumlashtirish o'lchovi emas. **first_loss** va **last_loss** yo'qotishning o'zgarishini beradi. Aniq bajarilgan qiymatlar natijalar ilovasida keltiriladi.

Xato va uning sababi

XOR muvaffaqiyatsiz bo'lsa, darhol yashirin qatlamni kattalashtirmang. Avval y shakli (4, 1) ekanini tekshiring: (4,) shakli tasodifiy kengayish natijasida (4, 4) xatoni hosil qilishi mumkin. Keyin gradient ishorasi, paket bo'yicha o'rtachalash va yangilash tartibini tekshiring.

1.7.3 Bulutda qayta tiklash: Google Colab, Hugging Face MNIST va Kaggle

Mahalliy `s01_numpy.py` skripti to'rtta qo'lda tuzilgan nuqtada ishlaydi; bu talabaga qo'lda gradient hisoblashning har bir qadamini kuzatish imkonini beradi. Ammo perseptron va faollashtirish tanlovining haqiqiy raqamli belgilarga qanday ta'sir qilishini ko'rish uchun kattaroq, haqiqiy ma'lumot kerak. Quyidagi cloudbox mahalliy GPU'siz, faqat Google hisobi bilan, Hugging Face Hub'dagi `mnist` datasetini yuklab, bitta neyron (perseptronga

yaqin chiziqli qatlam) va sigmoid/ReLU faollashtirishli kichik tarmoqni solishtirish tajribasini beradi.

Colab + Hugging Face: MNIST'da bitta neyron va XOR intuitsiyasi

Nega kerak. XOR misoli chiziqli ajratuvchining o'zligini to'rtta nuqtada ko'rsatadi; MNIST'dagi "0" va "1" raqamlarini ajratish esa xuddi shu g'oyani minglab haqiqiy piksel vektorida sinaydi. **Muhit.** Yangi Colab daftari o'chirib (<https://colab.research.google.com>), Runtime -> Change runtime type -> GPU tanlang – bu tajriba uchun GPU shart emas, lekin transformers/datasets kutubxonalari bilan tanishish uchun foydali. **Ma'lumot manbai.** Hugging Face Hub'dagi rasmiy mnist dataseti (joriy versiya va litsenziya tafsilotlarini <https://huggingface.co/datasets/mnist> sahifasidan tekshiring); bu klassik LeCun va h.k. to'plamining Hub'dagi ko'chirmasi.

```

1 !pip install -q datasets numpy
2 from datasets import load_dataset
3 import numpy as np
4
5 ds = load_dataset("mnist", split="train[:2000]")
6 # Faqat 0 va 1 raqamlarini olib, ikkilik tasniflash masalasiga aylantiramiz
7 mask = [l in (0, 1) for l in ds["label"]]
8 imgs = np.stack([np.array(im) for im, m in zip(ds["image"], mask) if m])
9 labels = np.array([l for l, m in zip(ds["label"], mask) if m])
10
11 X = imgs.reshape(len(imgs), -1) / 255.0      # (N, 784), [0,1] oralig'ida
12 y = labels.reshape(-1, 1).astype(np.float64) # (N, 1)
13
14 rng = np.random.default_rng(0)
15 w = rng.normal(0, 0.01, size=(X.shape[1], 1))
16 b = np.zeros((1, 1))
17 eta = 0.1
18 for epoch in range(50):
19     z = X @ w + b
20     a = 1 / (1 + np.exp(-z))      # sigmoid, bob matnidagi sigma(z)
21     grad_w = X.T @ (a - y) / len(X)
22     grad_b = np.mean(a - y)
23     w -= eta * grad_w
24     b -= eta * grad_b
25 acc = ((a >= 0.5) == y).mean()
26 print("O'qitish aniqligi:", acc)

```

Topshiriq. Yuqoridagi kodni Colab'da bajaring, so'ng eta qiymatini 0.01 va 1.0 ga o'zgartirib natijani yozing; XOR bo'limidagi kabi bitta chiziqli neyron nima uchun "0 va 1" juftini ajratishda muvaffaqiyatli, lekin vizual jihatdan yaqinroq raqam juftlarida (masalan "4 va 9") ancha yomonroq ishlashini kuzating va bir jumlada sababini tushuntiring. Natijani mahalliy s01_numpy.py chiqishidagi XOR aniqligi bilan solishtirib, ikkalasida ham "chiziqli ajratuvchining yetarliligi" savolini muhokama qiling. **Kaggle muqobili.** Xuddi shu masalani Kaggle'ning mashhur digit-recognizer

musobaqasi (<https://www.kaggle.com/c/digit-recognizer>) orqali ham qayta tiklash mumkin: Kaggle Notebooks ichida datasetni to'g'ridan-to'g'ri ulash, haftalik bepul GPU kvotasidan foydalanish va natijani `submission.csv` formatida yuklab, ochiq reyting jadvali bilan solishtirish mumkin.

Amaliy eslatma. Hugging Face Hub'dagi har bir datasetning sahifasida model/-dataset kartasi (dataset card) litsenziya va yig'ilish shartlarini ko'rsatadi; MNIST akademik tadqiqot uchun ochiq, ammo original ma'lumot yig'ilgan aholi tarkibi cheklangan bo'lib, undan chiqarilgan xulosalarni umumiy xat tanish tizimlariga to'liq umumlashtirib bo'lmaydi.

1.8 Zamonaviy kontekst (2026): bitta neyrondan poydevor modeligacha

Nima o'zgardi va nima o'zgarmadi?

2012-yildagi AlexNetdan [7] bugungi ko'p milliard parametrli modellargacha bo'lgan yo'lda uchta narsa o'zgarmadi: qatlam hali ham affin akslantirish, undan keyin nochiziqli funksiya qo'llanadi, va o'qitish differensiallanuvchi yo'qotishning gradienti bilan boshqariladi. O'zgargani — miqyos, ma'lumot va nochiziqlilikning aniq ko'rinishi. Shu sababli ushbu bobdagi $\sigma(w^\top x + b)$ formulasi eskirmagan: u bugungi eng katta modelning ham eng kichik qurilish g'ishtidir.

1.8.1 Darvozali nochiziqlilik: GELU dan SwiGLU ga

Yuqorida ko'rilgan ReLU, LeakyReLU va GELU bitta skalar kirishga bitta skalar funksiya qo'llaydi. Zamonaviy Transformer bloklarining oldinga uzatuvchi qismida boshqa oila — *darvozali chiziqli birlik* (gated linear unit) — keng tarqaldi. Uning umumiy ko'rinishi ikkita alohida proyeksiyani ko'paytirishga asoslanadi:

$$\text{GLU}_\phi(x) = (\phi(W_1x + b_1)) \odot (W_2x + b_2),$$

bu yerda $\odot$ — Hadamard (elementlar bo'yicha) ko'paytma, ϕ — skalar nochiziqlilik. ϕ sifatida $\text{swish}_\beta(z) = z \sigma(\beta z)$ olinsa, natija SwiGLU deb ataladi. $\beta = 1$ da

$$\text{swish}(z) = z\sigma(z), \quad \text{swish}'(z) = \sigma(z) + z\sigma(z)(1 - \sigma(z)).$$

Bu hosila GELU hosilasi $\Phi(z) + z\varphi(z)$ bilan bir xil tuzilishga ega: ikkalasi ham “o'tkazish ulushi + o'tkazish ulushining kirishga sezgirliigi” ko'rinishida. Farq shundaki, swish da o'tkazish ulushi $\sigma(z)$, GELU da esa $\Phi(z)$; σ va Φ bir-biriga yaqin S-shaklli funksiyalar bo'lgani uchun ikki faollashtirish amalda bir-biriga juda yaqin natija beradi.

Nima uchun ishlaydi. Oddiy $\phi(Wx + b)$ da har bir chiqish koordinatasi kirishning bitta chiziqli kombinatsiyasiga bog'liq. GLU da esa bitta shox *qancha* o'tkazishni, ikkinchi

shox *nimani* o'tkazishni belgilaydi. Bu ko'paytirishli o'zaro ta'sir bo'lib, faqat qo'shish va bitta nochiziqlilik bilan ifodalash qiyin bo'lgan funksiyalarni bir qatlamda ifodalash imkonini beradi.

Parametr hisobi va keng tarqalgan chalkashlik. Yashirin kenglik d va oraliq kenglik d_{ff} uchun oddiy FFN ikkita matritsa saqlaydi: $2dd_{\text{ff}}$ parametr. GLU oilasi uchta matritsa saqlaydi (W_1, W_2 va chiqish proyeksiyasi): $3dd_{\text{ff}}$. Shu sababli amalda parametr sonini teng ushlab turish uchun d_{ff} taxminan $2/3$ ga kamaytiriladi. Ikki variantni solishtirganda oraliq kenglikni tenglashtirish emas, *parametr yoki hisoblash byudjetini* tenglashtirish kerak — aks holda taqqoslash asossiz bo'ladi.

Faollashtirishni almashtirib, natijani xato talqin qilish

Kichik modelda ReLU ni GELU yoki SwiGLU ga almashtirib, validatsiya aniqligining 0,3% yaxshilanishini “SwiGLU ustun” deb yozish — eng keng tarqalgan xato. Bunday farq odatda seed tarqoqligidan kichik bo'ladi. To'g'ri protokol: kamida beshta seed, bir xil parametr byudjeti, medianani va tarqoqlikni birga keltirish. Agar farq tarqoqlikdan kichik bo'lsa, xulosa “bu sharoitda farq aniqlanmadi” bo'ladi, “teng” yoki “ustun” emas.

1.8.2 Chiziqli qatlamning zamonaviy cheklovlari

Bitta neyron darajasida ham bugungi amaliyotga tegishli uchta cheklov mavjud.

Birinchidan, kalibrlanganlik. Bob boshida ko'rsatilganidek, w va b ni bir xil songa ko'paytirish qaror chegarasini o'zgartirmaydi, lekin sigmoid chiqishini o'tkirlashtiradi. Katta modellarda bu muammo yo'qolmaydi, aksincha kuchayadi: yuqori aniqlikka ega model ham haddan tashqari ishonchli (overconfident) bo'lishi mumkin. Shuning uchun ishlab chiqarish tizimida faqat accuracy emas, kalibrlanganlik ham o'lchanadi.

Ikkinchidan, sonli barqarorlik. $\log \sigma(z)$ ni σ ni hisoblab, keyin logarifm olish orqali topish $|z|$ katta bo'lganda aniqlikni yo'qotadi. Barqaror shakl:

$$\log \sigma(z) = -\log(1 + e^{-z}) = -\text{softplus}(-z),$$

va $\text{softplus}(u) = \max(u, 0) + \log(1 + e^{-|u|})$ ko'rinishida hisoblanadi. Kutubxonalaridagi **BCEWithLogitsLoss** aynan shu shaklni ishlatadi; shu sababli sigmoidni qo'lda qo'llab, keyin oddiy BCE chaqirish tavsiya etilmaydi.

Uchinchidan, past aniqlikdagi hisoblash. Zamonaviy o'qitish ko'pincha **bfloat16** da bajariladi. Bu format **float32** bilan bir xil darajali diapazonga ega, lekin mantissasi qisqa. Natijada σ ning to'yingan sohasida $1 - \sigma(z)$ ayirmasi nolga aylanib qolishi mumkin. Bu bobdagi formulalar matematik jihatdan to'g'ri qolaveradi, lekin ularni to'g'ridan-to'g'ri past aniqlikda hisoblash xato beradi.

O'zingizni tekshiring

1. $\text{swish}(z) = z\sigma(z)$ uchun $z \rightarrow -\infty$ da limitni toping va uni ReLU bilan solishtiring.
2. $d = 512$, $d_{\text{ff}} = 2048$ bo'lsa, oddiy FFN va SwiGLU FFN parametrlari sonini yozing. SwiGLU ni teng byudjetga keltirish uchun d_{ff} qanday bo'lishi kerak?
3. Nima uchun $\log \sigma(z)$ ni to'g'ridan-to'g'ri hisoblash $z = -40$ da xavfli, $z = +40$ da esa xavfsiz?

1.8.3 O'lchanadigan mini-tajriba: faollashtirishlarni halol solishtirish

Ma'lumot. Hugging Face Hub'dagi `mnist` to'plami; train dan 10 000, validatsiya uchun 2000 kuzatuv ajratiladi.

Model. Bitta yashirin qatlam, kenglik 256; faqat faollashtirish o'zgaradi: ReLU, GELU, SwiGLU (oxirgisida kenglik $170 \approx \frac{2}{3} \cdot 256$ olinadi, parametr byudjeti tenglashtiriladi).

Metrika. Validatsiya top-1 aniqligi va kross-entropiya; beshta seed bo'yicha median va min–maks.

Kutilayotgan natija. Bunday hajmdagi MLP uchun validatsiya aniqligi odatda 0,96–0,98 oralig'ida bo'ladi; uch faollashtirish orasidagi farq ko'p hollarda seed tarqoqligi bilan bir tartibda chiqadi. Agar sizda faqat bitta seedda katta farq chiqsa, bu natija emas, shovqin.

Hisoblash byudjeti. CPU'da har bir seed uchun bir necha daqiqa; uchta faollashtirish $\times$ beshta seed Colab bepul rejimida GPU'siz ham bimalol bajariladi. Bu tajriba GPU kvotasini sarflashga arzimaydi.

Xatolar tahlili. Aralashuv matritsasida eng ko'p chalkashadigan juftlarni (odatda “4–9” va “3–5” turidagi juftlar) aniqlang va ular uchun xatoning faollashtirishga bog'liqligini alohida tekshiring. Umumiy aniqlik bir xil bo'lsa ham, xato taqsimoti farq qilishi mumkin — bu accuracy ko'rsatmaydi.

1.9 Neyron qarorini geometrik va statistik o'qish

1.9.1 Gipertekislikdan ehtimollik sirtiga

$w \neq 0$ bo'lsa, $w^\top x + b = 0$ qaror chegarasi gipertekislikdir. Nuqtaning chegaragacha ishorali masofasi $(w^\top x + b)/\|w\|$. Og'irlik va siljishni bir xil musbat songa ko'paytirish chegarani o'zgartirmaydi, lekin sigmoid ehtimollarini o'tkirlashtiradi. Demak, bir xil tasniflovchi chegara turli kalibrlanganlikka ega bo'lishi mumkin.

Misol 1.2 (Chegara teng, ishonch farqli). $z = 0.2$ uchun $\sigma(z) \approx 0.55$, $10z = 2$ uchun $\sigma(10z) \approx 0.88$. Har ikkala holatda threshold 0.5 bo'yicha qaror musbat. Ammo ehtimollik sifatida berilgan da'vo farqli. Accuracy bu farqni ko'rmaydi; kross-entropiya ko'radi.

Bu misol modelni faqat “to'g'ri/noto'g'ri” bilan o'qitish nega yetarli emasligini ko'rsatadi. Differensiallanuvchi yo'qotish qarorning to'g'riligi bilan birga logitning qanchalik qat'iy bo'lishini ham boshqaradi. Ehtimollik noto'g'ri sinfga qat'iy berilsa, kross-entropiya katta jarima beradi.

1.9.2 Belgi shkalasi gradientga ta'sir qiladi

Ikkita belgi bir xil mazmuniy ahamiyatga ega, lekin biri metr, ikkinchisi millimetrda bo'lsin. $\partial L / \partial w_j = \delta x_j$ sababli kattaroq birlikdagi belgi katta gradient beradi. Bu uning muhimroq ekanini emas, birlik tanlashning ta'sirini ko'rsatadi. Train o'rtachasi μ_j va standart og'ishi s_j bilan $\tilde{x}_j = (x_j - \mu_j) / (s_j + \epsilon)$ olish o'qitish geometriyasini yaxshilashi mumkin. Validatsiya va test ham aynan train statistikasi bilan o'zgartiriladi. Me'yorlashni butun to'plamda hisoblash test taqsimoti haqidagi axborotni oldindan olib kiradi. Xato kichik ko'rinsa ham, yakuniy baholashning mustaqilligi buziladi. Vaqt qatorida kelajak o'rtachasi o'tmishdan ma'lum emasligi ayniqsa yaqqol.

1.10 Ifodalash, optimallashtirish va umumlashtirish farqi

Bir model masalani ifodalay oladimi, algoritm shu parametrni topadimi va topilgan parametr yangi kuzatuvda yaxshi ishlaydimi — uchta alohida savol. XOR formulasi birinchisiga javob beradi. S1dagi o'qitish ikkinchisini berilgan seed va qadamda sinaydi. To'rtta nuqtani qayta tasniflash uchinchisiga mustaqil dalil bermaydi. Cheklangan hududdagi uzluksiz funksiyalarni yetarlicha keng tarmoq bilan yaqinlashtirish haqidagi teoremlar amaliy tanlanma hajmini, eng yaxshi arxitekturani yoki gradientning global yaqinlashishini bermaydi. “Universal approksimator” iborasini aynan shu cheklovlar bilan o'qish kerak. Masalan, million parametrli model to'rtta nuqtani yodlashi mumkin, lekin ma'lumotdagi simmetriyani tushunganini isbotlamaydi.

1.11 Auditoriya uchun hisoblangan mini-tajriba

Masala. Kirish $x = (1, 2)$; ikkita yashirin ReLU neyron $w_1 = (1, -1)$, $b_1 = 0$ va $w_2 = (1, 1)$, $b_2 = -1$. Chiqish $z = 2h_1 - h_2 + 0.5$ bo'lsin. Avval grafik orqali belgilar ma'nosini taxmin qiling, so'ng hisoblang. **Yechim.** Yashirin kirishlar -1 va 2 , aktivatsiyalar 0 va 2 . Chiqish logiti -1.5 , binar ehtimol taxminan 0.1824 . Birinchi neyron aynan shu nuqtada nol gradient oladi. Bu uning butun to'plamda o'chganini bildirmaydi; boshqa kirishda musbat bo'lishi mumkin. **Muqobil.** Tanh bilan birinchi yashirin qiymat $\tanh(-1)$ bo'lib nol emas. Lekin bu almashtirish chiqish shkalasini ham o'zgartiradi; faqat bitta nuqta natijasidan faollashtirish sifati haqida xulosa chiqarilmaydi.

1.12 S1 uchun mustaqil tajriba jurnali

Daftarda besh ustun ochiladi: faraz, o'zgartirilgan parametr, saqlangan sharoit, kuzatilgan natija va xulosa. Birinchi tajriba yashirin kenglikni 2 , 4 va 8 qiladi; ikkinchisi qadamni 0.02 , 0.2 va 2 qiladi. Ikkalasini birdan o'zgartirish natija sababini ajratishni qiyinlashtiradi. Har variantda bir xil boshlang'ich matritsalarni imkon qadar moslashtiring yoki seedlar

bo'yicha takrorlang. Talaba natija chiqishidan oldin “qadam juda katta bo'lsa, loss tebranishi mumkin” kabi tekshiriladigan faraz yozadi. Tajribadan keyin faqat eng chiroyli grafik emas, muvaffaqiyatsiz variantning sababi ham qayd qilinadi. Shu tartib keyingi murakkab tarmoqlarni tahlil qilish uchun poydevor yaratadi.

1.13 Muammo – Modellashtirish – Kod – Natija – Xulosa: XOR misolida to'liq zanjir

Ushbu bo'lim bob boshida keltirilgan besh bosqichli pedagogik zanjirni XOR masalasi misolida to'liq ko'rsatadi, chunki tarqoq holda berilgan nazariya, kod va natija talabaga ba'zan bog'liqsiz ko'rinishi mumkin.

Muammo. Ikkita ikkilik kirish uchun “aynan bittasi 1” shartini qanoatlantiruvchi qaror chegarasi qanday ko'rinishga ega, va bitta affin neyron unga yetarlimi?

Modellashtirish. Bob matnidagi teorema shartning affin funksiya bilan ifodalansligini isbotladi. Shu sababli model ikki qatlamli tarmoqqa kengaytiriladi: $h = \tanh(W_1x + b_1) \in \mathbb{R}^2$, $\hat{y} = \sigma(w_2^\top h + b_2)$. Yo'qotish sifatida binar kross-entropiya tanlanadi, chunki $\hat{y}$ ehtimol sifatida talqin qilinadi va bob matnida ko'rsatilgandek sigmoid aynan shu talqinga mos keladi.

Kod. `python/s01_numpy.py` skripti aynan shu modelni qo'lda hisoblangan gradientlar bilan o'qitadi; §1 bo'limidagi `\codefile` orqali kiritilgan kod bilan bir xil.

Natija. `results/s01.json` faylida saqlangan `first_loss` va `last_loss` qiymatlari yo'qotishning kamayishini ko'rsatadi; to'rtta XOR nuqtasi uchun yakuniy ehtimollar qaror chegarasining to'g'ri tomonida joylashadi.

Xulosa va tahlil. Natija dastlabki farazni tasdiqlaydi: ikki yashirin neyron XOR chegarasini ifodalash uchun yetarli. Ammo bu xulosa faqat to'rtta nuqta uchun, berilgan boshlang'ich qiymat va qadam uchun amal qiladi – boshqa seedda tarmoq mahalliy minimumda qolib, bitta nuqtani noto'g'ri tasniflashi mumkin (shuning uchun §1 oxiridagi “Yuqori” darajali topshiriq beshta seed bilan qayta tekshirishni talab qiladi). Ifodalash imkoniyati (ikki neyron yetarli) bilan optimallashtirish muvaffaqiyati (aynan shu seedda topilgan yechim) o'rtasidagi farqni ajratib ko'rsatish – ushbu zanjirning asosiy pedagogik natijasidir.

1.14 Bulutli platformada birinchi tajriba

Yuqoridagi zanjirning “Kod” bosqichi mahalliy Python muhitini talab qiladi. Talabaning shaxsiy kompyuteri mos kelmasa yoki kutubxona o'rnatishda muammo chiqsa, xuddi shu tajriba brauzer orqali bepul bulutli muhitda bajarilishi mumkin. Quyidagi tartib S1 tajribasini hech narsa o'rnatmasdan takrorlashga imkon beradi.

Google Colab: S1 tajribasini brauzerda bajarish

1. `colab.research.google.com` sahifasida “New notebook” tugmasini bosing. Birinchi bobdagi tajriba faqat NumPy talab qiladi, shu sababli **Runtime** → **Change runtime type** bo'limida CPU qoldirish yetarli; GPU bu yerda tezlik bermaydi va kvotani behuda sarflaydi. 2. Zarur paketlarni o'rnatish:

```
1 !pip install -q numpy matplotlib
```

3. `python/s01_numpy.py` faylini chap paneldagi “Files” bo'limiga yuklang, so'ng katakchada `!python s01_numpy.py` buyrug'ini bajaring. Chiqishdagi `first_loss` va `last_loss` qiymatlarini `results/s01.json` dagi mahalliy qiymatlar bilan solishtiring. 4. Colab seansi uzilganda fayllar o'chadi. Natijani saqlash uchun daftarni Google Drive'ga ulang yoki natija faylini qo'lda yuklab oling.

Hugging Face va Kaggle: XOR dan real ma'lumotga o'tish

XOR to'rtta nuqtadan iborat sun'iy masala; undan olingan xulosa real taqsimotga avtomatik ko'chmaydi. Shu sababli aynan shu tarmoq mantig'ini kattaroq, haqiqiy to'plamda sinab ko'rish tavsiya etiladi.

```
1 from datasets import load_dataset
2
3 ds = load_dataset("mnist")
4 print(ds["train"].features)
5 image = ds["train"][0]["image"]
6 label = ds["train"][0]["label"]
```

1. Hugging Face Hub'dagi `mnist` to'plami yuqoridagi qatorlar bilan yuklanadi (`!pip install -q datasets` talab qilinadi). To'plam kartasida litsenziya va tuzilish ko'rsatilgan; joriy versiyani Hub sahifasidan tekshiring. 2. Bitta chiqish neyroni bilan faqat ikki sinfni (masalan 0 va 1 raqamlarini) ajrating. Bu bob matnidagi sigmoid va binar kross-entropiya mantig'ini o'zgartirmasdan real ma'lumotga qo'llash demakdir. 3. Kaggle'dagi `digit-recognizer` musobaqasi xuddi shu masalaning baholanadigan variantidir. Notebook oching, **Add Input** orqali musobaqa ma'lumotini ulang va o'z tasniflovchingizni yuboring. Haftalik GPU kvotasi cheklangan, lekin bu masala uchun CPU yetarli. 4. Model va to'plam kartasini o'qish odatini boshidanoq shakllantiring: ochiq to'plam bepul bo'lishi uning har qanday maqsadda ishlatilishiga ruxsat berilganini anglatmaydi.

Bulutli natija mahalliy natijani almashtirmaydi. Hisobotda qaysi muhitda, qaysi kutubxona versiyasi bilan va qaysi tanlanma hajmida ishlanganini har doim qayd eting; aks holda ikki natija o'rtasidagi farqning sababi aniqlanmay qoladi.

1.15 Bob xulosasi va asosiy tushunchalar

Asosiy xulosa. Sun'iy neyron — affin akslantirish va undan keyingi nochiziqlilik. Bu ikki bosqichning kompozitsiyasi eng katta zamonaviy modellarning ham qurilish g'ishtidir; o'zgargani miqyos va nochiziqlilikning aniq shakli, printsip emas.

Nimani o'zlashtirish kerak.

- Bir nechta affin qatlam kompozitsiyasi affin bo'lib qoladi — nochiziqlilik shuning uchun zarur.
- XOR bitta affin neyron bilan ifodalanmaydi; ikki yashirin neyron yetarli.
- Faollashtirish tanlovi: to'yinish, gradient uzatish va hisoblash xarajati orasidagi almashinuv.
- Boshlashning dispersiya hisobi: $\text{Var}(w) \approx 1/d$, ReLU uchun $\approx 2/d$.
- Kalibrlanganlik aniqlikdan ajralgan tushuncha: bir xil qaror chegarasi turli ishonch berishi mumkin.

Keyingi bobga bog'lanish. Bu yerda gradient qo'lda chiqarildi. Keyingi bob shu hisobni ixtiyoriy chuqurlikdagi tarmoq uchun tizimli ravishda bajaruvchi mexanizmni — hisoblash grafi va orqaga tarqalishni — beradi.

1.16 Mustahkamlash topshiriqlari

Topshiriq 1.1 (O'rta). $x = (1, 2, -1)$, $w = (2, -1, 0.5)$, $b = 1$ uchun beshta faollashtirish chiqishi va hosilasini hisoblang. Qo'lda va kodda natijalarni taqqoslang.

Topshiriq 1.2 (O'rta). Uch affin qatlam kompozitsiyasi affin ekanini isbotlang; umumiy og'irlik va siljishni yozing.

Topshiriq 1.3 (O'rta). OR va AND uchun perseptron og'irliklarini qo'lda toping. Chegaraga eng yaqin nuqta masofasini hisoblang.

Topshiriq 1.4 (O'rta). Sigmoidning eng katta hosilasi $1/4$ ekanini isbotlang. Maksimum qaysi kirishda olinadi?

Topshiriq 1.5 (O'rta). $B = 16$, $d = 20$, $h = 7$ uchun paket modelining har bir tensor shakli va parametrlar sonini aniqlang.

Topshiriq 1.6 (Yuqori). Ikki ReLU yashirin neyron orqali XORni to'rtta nuqtada aniq ifodalang. Og'irliklarni o'qitishsiz yozing.

Topshiriq 1.7 (Yuqori). S1 ni beshta seed bilan bajaring; yaqinlashish tezligining medianasi va diapazonini yozing. Muvaffaqiyatsizlik bo'lsa saqlang.

Topshiriq 1.8 (Yuqori). $f(z) = z\Phi(z)$ hosilasi manfiy bo'ladigan intervalni sonli toping. GELU monotonligi haqidagi xulosani tekshiring.

Topshiriq 1.9 (Yuqori). He boshlashdagi mustaqillik farazini buzadigan bog'langan kirish yarating; qatlam dispersiyasini nazariya bilan taqqoslang.

Topshiriq 1.10 (Yuqori). ReLU neyronini o'chiradigan tajriba tuzing va qadamni kamaytirish hamda LeakyReLU bilan almashtirishni alohida taqqoslang.